PD Dr. Ilche Georgievski
Service Computing Department
Institute of Architecture of Application Systems
University of Stuttgart

Master thesis

Towards an Autonomous Lifecycle Agent for PDDL Domain Modelling

Context and Motivation

Automated planning is a foundational subfield of artificial intelligence concerned with the synthesis of action sequences that transform a given initial state into a desired goal state [1]. The Planning Domain Definition Language (PDDL) [2] is the de facto standard formalism for specifying planning problems, defining typed objects, predicates, and action schemas with explicit preconditions and effects.

Constructing a correct, complete, and efficient PDDL domain model is a laborious and error-prone process [3]. In practice, domain engineering follows an iterative *lifecycle* that extends far beyond initial model creation: domain engineers elicit semantics from natural-language requirements, generate PDDL action schemas, validate models with tools such as VAL [4], diagnose issues, repair faulty preconditions or effects, and evolve models as system requirements change. Each activity feeds into the next, and failures at later stages routinely demand revisions at earlier ones. Some works that explore aspects of this lifecycle include [5] and [6].

Advancements of Large Language Models (LLMs) has raised expectations for their role in automating knowledge-intensive engineering tasks [7]. Their application to PDDL domain engineering is a natural and timely direction. A recent comprehensive survey systematically maps this growing landscape, covering over 80 works on LLM-driven planning model construction across generation, editing, and benchmarking—confirming both the maturity of the field and the absence of a unified lifecycle approach [8]. More broadly, the emergence of agentic AI—systems in which an LLM serves not merely as a question-answering tool but as the reasoning core of an *autonomous agent*—has fundamentally expanded what LLM-based automation can achieve. An agentic system perceives its environment, maintains state across multiple steps, selects and invokes external tools, and pursues a goal over an extended horizon without requiring human intervention at each decision point [9]. This paradigm shift, from single-shot inference to sustained autonomous operation, is the architectural foundation of the agent to be developed in this thesis project.

The motivation for this research rests on a central observation: *PDDL domain engineering is a lifecycle, not a generation task*, and the automation tools deployed against it should reflect this structure. Existing LLM-based approaches treat planning domain modelling as a collection of isolated subtasks, each addressed by a separate, specialised system. What is missing is an **autonomous agent** that orchestrates the full lifecycle—one that can enter the loop at any stage, invoke the right tools, interpret their output, and iterate until convergence, without

human steering between steps. This is precisely the class of problem for which agentic AI is designed, and it constitutes the research opportunity addressed by this thesis project.

State of the Art

A comprehensive survey maps the landscape of LLM-driven automated planning model construction, organising approximately 80 works into model generation, model editing, and benchmarking [8]. Notably, the survey identifies no system that treats domain engineering as a unified, end-to-end lifecycle agent, confirming the gap this project addresses. Within this landscape, some lines of work are as follows. LLM+P [10] demonstrates that LLMs can translate natural-language task descriptions into PDDL problem files for downstream solving, functioning as a language interface rather than a domain engineering agent. Silver et al. [11] extend this to domain generalisation via iterative prompting and automated debugging loops. Oswald et al. [12] and related work explore GPT-4 for PDDL domain generation from natural language. Auto-PlanBench [13] takes the inverse direction: it automatically converts PDDL benchmarks into natural-language descriptions for evaluating LLM planners, providing useful infrastructure for benchmark construction but not addressing domain engineering or repair.

Closer to the problem at hand, several recent systems introduce *iterative refinement loops* in which a planner or validator provides feedback to an LLM. Smirnov et al. [14] perform automated consistency checking during PDDL generation, feeding error descriptions back to the LLM in a correction loop, with plan reachability verified by Fast Downward as a final step. An environment-interaction approach uses planner feedback to progressively refine PDDL domains via a tree-search over candidate files[15]. Addressing a related but distinct problem, Chen et al. [16] introduce LASP, a framework that uses an LLM to diagnose and repair an incomplete PDDL domain at execution time: when a robot encounters a physical execution failure, LASP identifies the missing precondition or object responsible and incrementally updates the knowledge base. While LASP demonstrates LLM-guided diagnosis and model repair within a closed loop, it is fundamentally a runtime robotics system — feedback comes from physical world observations, not planner output — and its scope is restricted to missing preconditions and missing objects in a single kitchen domain. It does not address offline domain engineering or broader lifecycle of domain modelling. Most recently, La Malfa et al. [17] present a fully automated, end-to-end multi-agent agentic framework that dynamically orchestrates specialised LLM agents to construct and iteratively refine PDDL models, with a symbolic planner guaranteeing correctness of the final output.

In the hierarchical planning setting, Lutalo and Bercher [18] address repair of TO-HTN domains with missing actions by translating the task into a context-free grammar repair problem and using an LLM to identify and insert relevant actions, guided by a plan known to be valid for the repaired model. While this demonstrates the feasibility of LLM-assisted domain repair, the approach is specific to one fault class (missing actions) in the HTN formalism, assumes a reference plan as input, and does not extend to classical PDDL or to the broader lifecycle activities addressed by UPDEAgent.

Despite this progress, a critical gap remains. Existing iterative systems are scoped to the *generation and syntactic repair* of PDDL models: they iterate until a plan is found or errors

are resolved, but do not address the broader domain engineering lifecycle. None handle domain evolution—extending an existing, previously valid domain with new action schemas without introducing regressions—nor do they provide explicit fault localisation: identifying *why* a domain fails rather than simply detecting *that* it fails. Furthermore, no existing work provides a benchmark spanning all lifecycle activities, making it impossible to evaluate and compare systems on the full range of engineering tasks that practitioners actually perform.

This thesis project does not claim priority on the feedback loop itself. Its novelty is fourfold and distinct from these contributions:

1. It introduces the lifecycle model for domain model engineering. Formalising the activities, their typed inputs/outputs, and dependency structure is likely new and would provide the theoretical grounding for the remaining contributions and future work.

2. None handle the full domain engineering lifecycle: for instance, **domain evolution**—extending an existing domain without introducing regressions—and **fault localisation**—reasoning about *why* a domain fails rather than merely detecting *that* it fails—remain unaddressed. The scope of the agent and benchmark developed in this project will be determined by the lifecycle model, but these gaps illustrate the class of problem that lifecycle-aware approach is designed to handle.

3. It addresses *offline* domain engineering as a full lifecycle. Offline, pre-deployment domain engineering is the primary bottleneck for practitioners, and no agentic system addresses it.

4. It introduces PDDLLifeCycleBench, the first benchmark evaluating LLM-based approaches across all lifecycle activities, enabling reproducible comparison. Existing benchmarks evaluate plan generation given a domain, not domain engineering across repair, diagnosis, and evolution tasks.

Problem Statement and Main Research Question

The problem addressed by this project is that *PDDL domain engineering is treated as a generation task when it is, in fact, a multi-activity lifecycle requiring autonomous, iterative orchestration.* No existing system provides an autonomous agent capable of managing this lifecycle end-to-end, and no benchmark exists to evaluate such a system across the full range of lifecycle activities.

Main Research Question: Can an autonomous LLM-based agent that orchestrates the full PDDL domain engineering lifecycle—integrating a classical planner as an active feedback oracle—outperform single-step generation baselines on a comprehensive lifecycle benchmark, and successfully handle complex multi-activity scenarios that single-step approaches cannot address?

Sub-questions:

- What are the distinct activities of the PDDL domain engineering lifecycle, their typed inputs and outputs, and the dependency structure between them?

- How should an autonomous agent reason about the layered feedback signals produced during domain validation and planning (e.g., covering parse errors, type and semantic violations, grounding failures, and solvability outcomes) to triage faults and select appropriate diagnostic and repair strategies for each signal class?
- What prompt engineering strategies (e.g., chain-of-thought, structured output, diagnostic scaffolding) most effectively support each lifecycle activity within an agentic loop?
- What classes of PDDL fault present the greatest challenge for autonomous diagnosis and repair, and what does this reveal about the limits of current LLM reasoning over formal state-transition models?

Planned Methodology

The project follows a design science and experimental research methodology, combining system design and implementation with empirical evaluation. The study is mixed-methods: qualitative analysis of the lifecycle model and prompt design is combined with quantitative experimental evaluation of agent performance. The work proceeds in five phases:

- Literature review and lifecycle analysis: a systematic review of PDDL domain engineering tools, including LLM-based ones, and agentic AI frameworks. This analysis should produce a formal lifecycle activity model specifying the key activities (e.g., elicitation, generation, validation, diagnosis, repair, evolution), their typed inputs and outputs, and their dependency structure. The lifecycle model will also inform a scoping decision: not all activities need to be addressed by the agent and benchmark in equal depth, and the student is expected to prioritise activities based on their centrality to the lifecycle, tractability, and coverage in existing work.
- Construction of PDDLLifeCycleBench. Some relevant sources for the construction include (i) classical benchmarks from the International Planning Competitions (IPCs), such as Blocksworld, Logistics, Rovers, Depots, and Satellite; (ii) systematically flawed benchmark domains for diagnosis and repair tasks; and (iii) natural-language domain descriptions paired with reference PDDL for elicitation and generation tasks. Evaluation metrics can be defined at this stage and may include plan validity rate, repair success rate, evolution correctness, iteration efficiency, fault localisation accuracy, etc.
- Agent design and implementation. The agent should be able to maintain lifecycle state, autonomously select the next activity, have access to suitable tools, such as VAL and a planner, invoke such a tool when appropriate, maintain the current domain, repair history, etc.
- Experimental evaluation. The developed agent should be evaluated on the developed PDDLLifeCycleBench against some baselines, such as zero-shot LLM generation and ablated variants of the developed agent (e.g., loop disabled or diagnosis module replaces by a generic retry prompt). Results are analysed quantitatively across metrics and qualitatively via error analysis to characterise remaining failure modes.

Expected Outcomes

This thesis project is expected to deliver the following contributions:

- A formal lifecycle model for PDDL domain engineering. This constitutes a conceptual contribution bridging the AI planning and agentic AI literatures. By formalising the lifecycle activities, their inputs and outputs, and their interdependencies, the model provides a reusable theoretical framework for future research in AI-assisted domain engineering.
- An autonomous domain engineering agent.
- PDDLLifeCycleBench as a reusable evaluation benchmark. The benchmark enables reproducible comparison of future LLM-based domain engineering approaches across all lifecycle activities.
- Empirical insight into LLM limitations for formal PDDL reasoning. The error analysis is expected to reveal systematic patterns in how LLMs fail at autonomous PDDL diagnosis and repair. Those findings will inform future work on neurosymbolic integration and have broader implications for any domain requiring autonomous LLM reasoning over formal state-transition models.
- If results are sufficiently strong, a workshop or conference paper submission can be submitted on the basis of the thesis.

Licensing

Concepts and implementation produced in this project must be compatible and published under the MIT license.

Keywords: PDDL domain engineering, agentic AI, autonomous agent, LLM-based automation, AI planning lifecycle

References

[1] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: Theory and Practice*. Morgan Kaufmann, 2004.

[2] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins, "PDDL—the planning domain definition language," Yale Center for Computational Vision and Control, Tech. Rep. CVC TR-98-003 / DCS TR-1165, 1998.

[3] I. Georgievski, "Engineering ai planning systems," June 2025. [Online]. Available: https://doi.org/10.18419/opus-16736

[4] R. Howey, D. Long, and M. Fox, "VAL: Automatic plan validation, continuous effects and mixed initiative planning using PDDL," in *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, 2004, pp. 294–301.

[5] J. R. Silva, J. M. Silva, and T. S. Vaquero, "Formal knowledge engineering for planning: Pre and post-design analysis," in *Knowledge Engineering Tools and Techniques for AI Planning*, M. Vallati and D. E. Kitchin, Eds. Springer, 2020, pp. 47–65.

[6] I. Georgievski, "Software development lifecycle for engineering ai planning systems," in *International Conference on Software Technologies*, 2023, pp. 751–760. [Online]. Available: https://www.scitepress.org/Link.aspx?doi=10.5220/0012149100003538

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[8] M. Tantakoun, C. Muise, and X. Zhu, "Llms as planning formalizers: A survey for leveraging large language models to construct automated planning models," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 25 167–25 188.

[9] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[10] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "LLM+P: Empowering large language models with optimal planning proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[11] T. Silver, S. Dan, K. Srinivas, J. B. Tenenbaum, L. P. Kaelbling, and M. Katz, "Generalized planning in PDDL domains with pretrained large language models," in *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, 2024, pp. 20 256–20 264.

[12] J. Oswald, K. Srinivas, H. Kokel, J. Lee, M. Katz, and S. Sohrabi, "Large language models as planning domain generators," in *Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS)*, 2024, pp. 423–431.

[13] K. Stein, D. Fišer, J. Hoffmann, and A. Koller, "AutoPlanBench: Automatically generating benchmarks for LLM planners from PDDL," in *Proceedings of the Workshop on Bridging the Gap Between AI Planning and Reinforcement Learning (PRL), ICAPS*, 2024.

[14] P. Smirnov, F. Joublin, A. Ceravola, and M. Gienger, "Generating consistent PDDL domains with large language models," *arXiv preprint arXiv:2404.07751*, 2024.

[15] S. Mahdavi, R. Aoki, K. Tang, and Y. Cao, "Leveraging environment interaction for automated PDDL translation and planning with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[16] G. Chen, L. Yang, R. Jia, Z. Hu, Y. Chen, W. Zhang, W. Wang, and J. Pan, "Language-augmented symbolic planner for open-world task planning," in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.

[17] E. La Malfa, P. Zhu, S. Marro, S. Bernardini, and M. Wooldridge, "An end-to-end planning framework with agentic LLMs and PDDL," *arXiv preprint arXiv:2512.09629*, 2025.

[18] D. Lutalo and P. Bercher, "Automated repair of totally-ordered hierarchical task network domains via context-free grammars with large language model support," in *Proceedings of the 40th AAAI Conference on Artificial Intelligence*, 2026.

Supervisor

PD Dr. Ilche Georgievski
ilche.georgievski@iaas.uni-stuttgart.de

Examiner

PD Dr. Ilche Georgievski